

Санкт-Петербургский государственный университет

Кафедра системного программирования

Группа 24.М41-мм

Михайлов Михаил Дмитриевич

Обнаружение разладки в контроле перегрузки сети

Отчёт по преддипломной практике
в форме «Решение»

Научный руководитель:
доцент кафедры системного программирования, к. ф.-м. н. Гориховский В. И.

Санкт-Петербург
2026

Содержание

Введение	3
Обзор предметной области	5
Задача обнаружения разладки и метрики качества	5
Инструменты для сравнения CPD-алгоритмов	6
CPD в сетевых задачах	7
Постановка задачи	10
Глава 1. Систематический обзор методов обнаружения раз- ладки, применяемых в сетевых задачах	11
1.1. Методика поиска источников	11
1.2. Выводы из обзора	12
Глава 2. Архитектура программного стенда	13
2.1. Требования к программному стенду	14
2.2. Общая архитектура	15
2.3. Архитектура слоя data	16
2.4. Архитектура генератора синтетических данных	17
2.5. Архитектура слоя core	18
2.6. Архитектура слоя analysis	18
2.7. Архитектура слоя benchmarking	19
Глава 3. Особенности реализации программного стенда .	22
3.1. Используемые технологии	22
3.2. Воспроизводимость экспериментов	22
3.3. Оптимизированные сценарии для онлайн алгоритмов .	23
Глава 4. Прототип специализированного алгоритма обна- ружения разладки	24
4.1. Общий подход	24
4.2. Реализация прототипа	28
Глава 5. Экспериментальное сравнение	32
5.1. Постановка эксперимента	32
5.2. Результаты эксперимента	34
5.3. Обсуждение результатов	35
Заключение	38
Приложение А. Подробные сведения о рассмотренных CPD-алгоритмах	39
Приложение В. Архитектура генератора синтетических данных	43

Введение

Современные транспортные протоколы TCP и QUIC используют алгоритмы контроля перегрузки (Congestion Control Algorithms, CCA), чтобы предотвращать перегрузку сети и одновременно не оставлять доступную пропускную способность неиспользованной. Для этого CCA должен своевременно реагировать на изменения состояния сети. Чем раньше обнаружено изменение в транспортных метриках, тем раньше алгоритм может адаптировать поведение отправителя.

Одним из подходов к поиску таких изменений является обнаружение разладки (Change Point Detection, CPD) — поиск момента, в который изменяются статистические свойства временного ряда. В сетевой задаче таким рядом может быть, например, задержка, пропускная способность, относительная односторонняя задержка или доля потерянных пакетов. Использование CPD в сетевых задачах уже встречается в литературе: например, в TCP-NeuRoc online-обнаружение разладки применяется совместно с обучением с подкреплением для адаптации TCP congestion control [5], а в RoME-QCD CUSUM-подход используется для мониторинга 5G-сетей по односторонней задержке [7].

При этом существующие методы часто плохо применимы к транспортным метрикам QUIC напрямую. Многие классические алгоритмы предполагают независимость наблюдений, нормальность распределений или заранее заданный вид изменения. Для сетевых данных эти предположения могут нарушаться: состояние сети наблюдается косвенно, метрики зависят друг от друга, а эталонная разметка моментов изменения обычно отсутствует. Поэтому разработка специализированного CPD-алгоритма требует не только выбора метода, но и создания программного стенда для подготовки данных, запуска алгоритмов и воспроизводимого сравнения результатов.

Работа выполняется в рамках совместного проекта СПбГУ и Huawei¹, направленного на исследование обнаружения изменений в транспортных метриках QUIC. В настоящей работе разрабатываются программный

¹Проект №TC20250703044

стенд для сравнения CPD-алгоритмов и прототип специализированного подхода к обнаружению разладки на базе библиотеки PySATL-CPD.

Обзор предметной области

Задача обнаружения разладки и метрики качества

Обнаружение разладки (change point detection, CPD) — задача поиска моментов времени, в которых изменяются статистические свойства наблюдаемого процесса [3, 10]. Для временного ряда это означает переход от одного режима наблюдений к другому. Изменение может выражаться в сдвиге среднего значения, изменении дисперсии, корреляционной структуры, сезонности или распределения. Различают offline- и online-постановки задачи. В offline-постановке алгоритм имеет доступ ко всему временному ряду. В данной работе рассматривается online-постановка: данные поступают последовательно, а будущие значения ряда недоступны.

Оценка online-алгоритмов включает факт обнаружения эталонной разладки, задержку сигнала и число ложных срабатываний. Для фиксированного порога алгоритма задаются числа истинно положительных обнаружений TP , ложных срабатываний FP и пропущенных разладок FN . На их основе вычисляются стандартные метрики:

$$Precision = \frac{TP}{TP + FP}, \quad (1)$$

$$Recall = \frac{TP}{TP + FN}, \quad (2)$$

$$F_1 = 2 \cdot \frac{Precision \cdot Recall}{Precision + Recall}. \quad (3)$$

Метрика $Precision$ задаёт долю сигналов, соответствующих эталонным разладкам, $Recall$ — долю обнаруженных эталонных разладок, а F_1 — их гармоническое среднее.

Для каждой эталонной разладки задержка определяется как расстояние от момента разладки до первого корректного сигнала алгоритма в допустимом окне обнаружения. Средняя задержка обнаружения, или

ADD (Average Detection Delay), вычисляется как

$$ADD = \frac{1}{K} \sum_{i=1}^K delay_i, \quad (4)$$

где K — число эталонных разладок, а $delay_i$ — задержка обнаружения i -й разладки.

Метрики зависят от порога алгоритма. Пороговые параметры разных методов имеют разный смысл, поэтому сравнение при одинаковом численном значении порога некорректно. Для порогонезависимого сравнения используется Precision–Recall-кривая, а площадь под этой кривой обозначается как PR-AUC:

$$PR-AUC = \int_0^1 Precision(Recall) dRecall. \quad (5)$$

Для анализа ложных срабатываний используется характеристика ARL (Average Run Length). В задачах обнаружения разладки под ARL понимается среднее число наблюдений до первого ложного срабатывания в режиме без разладки:

$$ARL = \mathbb{E}_\infty T, \quad (6)$$

где T — момент первого сигнала алгоритма, а индекс ∞ означает, что математическое ожидание берётся при отсутствии разладки. Итого online-алгоритм оценивается по полноте обнаружения, числу ложных срабатываний и задержке реакции.

Инструменты для сравнения CPD-алгоритмов

Для разработки CPD-алгоритмов важна не только реализация самих методов, но и инфраструктура сравнения: единый формат данных, единый способ запуска алгоритмов, вычисление метрик качества и воспроизводимое хранение результатов. PySATL-CPD содержит ядро для запуска методов CPD и экспериментальное окружение для изучения эффективности алгоритмов². Turing Institute CPD benchmark содержит

²<https://github.com/PySATL/PySATL-CPD-Module>

набор реальных временных рядов с независимыми аннотациями и реализует сравнение алгоритмов на единой базе [2]. PySATL-CPD не является специализированным стендом для сетевых метрик QUIC/BBR. Turing benchmark ориентирован на временные ряды из разных предметных областей и не учитывает требования online-сравнения сетевых алгоритмов по задержке обнаружения и ложным срабатываниям.

Для настоящей работы ключевым является не только наличие общего CPD-бенчмарка, но и применимость методов обнаружения разладки к сетевым метрикам транспортного протокола.

CPD в сетевых задачах

В сетевых задачах временные ряды возникают при наблюдении метрик транспортного протокола или сетевой инфраструктуры: RTT, OWD, пропускной способности, числа потерянных пакетов, размера окна перегрузки и других величин. Разладка в таком ряде может соответствовать изменению доступной пропускной способности, росту задержки, перегрузке, изменению маршрута или переходу сети в другой режим работы.

Методы обнаружения разладки применяются к сетевым данным и встраиваются в алгоритмы управления сетью. В TCP-NeuRoc online-обнаружение разладки используется совместно с агентом обучения с подкреплением для адаптации TCP congestion control к новым режимам сети [5]. В RoME-QCD рассматривается обнаружение изменений сетевых характеристик, в частности односторонней задержки, при ограниченной частоте измерений [7]. Эти работы не дают готового решения для online-анализа метрик QUIC/BBR и сравнения алгоритмов в единых условиях.

Универсальный набор размеченных сетевых данных для сравнения CPD-алгоритмов отсутствует. В сетевых данных моменты изменения режима не всегда наблюдаемы напрямую. Сетевые метрики могут быть шумными, зависимыми, многомерными, непрерывными и дискретными. Для разработки и проверки алгоритмов обнаружения разладки требуются данные с известными моментами изменения. В работе [9] рассматривается разметка изменений RTT и путей в сети, однако такая

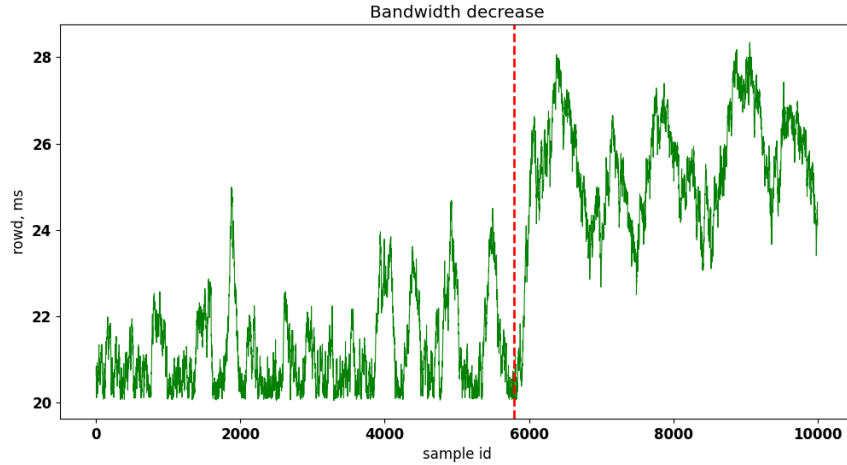


Рис. 1: Пример разладки, хорошо различимой по метрике *relative one-way delay*

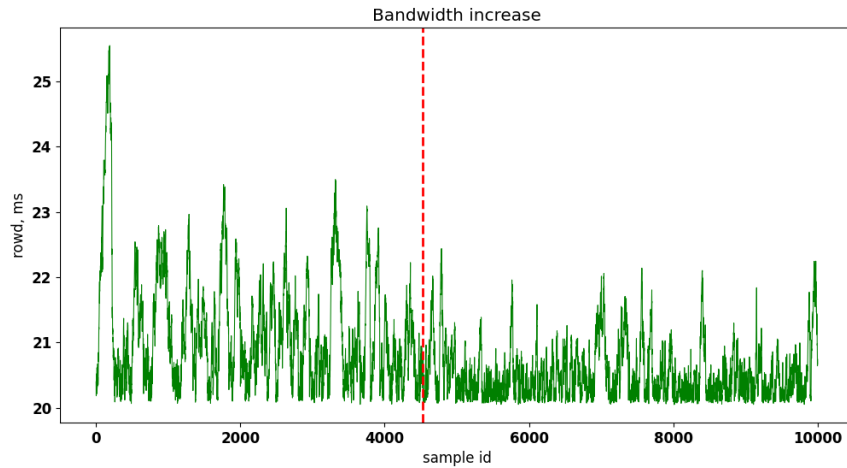


Рис. 2: Пример разладки, слабо выраженной по метрике *relative one-way delay*

постановка отличается от задачи генерации согласованных транспортных метрик QUIC/BBR.

Платформы для тестирования QUIC работают с реальными реализациями протокола, но ориентированы на проверку производительности, а не на генерацию размеченных CPD-датасетов. Сетевые симуляторы, такие как ns-3 и Mininet, позволяют задавать параметры каналов, очередей и задержек. При этом они могут требовать отдельного тестового стенда, не обеспечивать разметку моментов разладки или не удовлетворять лицензионным и инфраструктурным ограничениям проекта.

Общие генераторы синтетических временных рядов основаны на случайных блужданиях, авторегрессионных моделях, генеративных нейронных сетях и других подходах. Они позволяют получать ряды с заданными статистическими свойствами, но не учитывают зависимость между RTT, OWD, пропускной способностью и потерями, сочетание непрерывных и дискретных величин, а также влияние алгоритма контроля перегрузки на наблюдаемые значения. Поэтому для экспериментальной оценки требуется программный стенд, который единообразно запускает алгоритмы, учитывает эталонную разметку, строит метрики качества, воспроизводит эксперименты и позволяет получать специализированные синтетические данные. Выявленные ограничения существующих методов и инструментов определяют цель и задачи настоящей работы.

Постановка задачи

Целью работы является разработка прототипа и экспериментальная оценка алгоритма обнаружения разладки, специализированного под транспортные метрики QUIC. Для достижения этой цели были поставлены следующие задачи:

- а) провести систематический обзор методов обнаружения разладки в сетевых данных и отобрать baseline-методы для сравнения;
- б) спроектировать и реализовать программный стенд для подготовки данных, в том числе синтетических, и запуска CPD-алгоритмов;
- в) разработать и реализовать прототип алгоритма обнаружения разладки, специализированного под данные QUIC;
- г) провести экспериментальное сравнение разработанного прототипа с baseline-методами на синтетических данных.

Глава 1. Систематический обзор методов обнаружения разладки, применяемых в сетевых задачах

В рамках проекта был выполнен систематический обзор работ, в которых методы обнаружения разладки применяются к сетевым данным. Полный текст обзора находится под ограничениями NDA, а авторские права на него принадлежат Huawei. Поэтому в настоящем разделе приведены только раскрываемые сведения: общая методика поиска, перечень выявленных методов и выводы, использованные при выборе алгоритмов для дальнейшего сравнения.

Патентный поиск, поиск наборов данных и поиск готовых реализаций были выполнены другими участниками проекта и не относятся к результатам настоящей работы. В данном разделе рассматривается только часть обзора, посвящённая методам обнаружения разладки в сетевых задачах, выполненная автором настоящей работы.

1.1. Методика поиска источников

Поиск был направлен на публикации, в которых методы обнаружения разладки используются в задачах анализа сетевого трафика, сетевой оптимизации или контроля перегрузки. Основным интерес представляли работы, где методы применялись к временным рядам сетевых метрик или рассматривались как часть решения сетевой задачи.

Для поиска источников использовалась система GOOGLE SCHOLAR, а также репозитории ARXIV и IEEE Xplore. В обзоре рассматривались три группы источников:

- тематические статьи, посвящённые применению методов обнаружения разладки к сетевым данным;
- обзорные статьи по методам обнаружения разладки, не обязательно в контексте сетевых задач;
- экспериментальные исследования, в которых сравнивались алгоритмы обнаружения разладки.

Для каждой категории источников были сформулированы поис-

ковые запросы. В запросах использовались ключевые слова связанные с обнаружением разладки, сетевым трафиком, сетевой оптимизацией, TCP и QUIC. Сводная статистика поиска и отбора источников приведена в таблице 1.

Таблица 1: *Статистика по источникам для систематического обзора*

Вид источника	Просмотрено	Отобрано
Тематические статьи	> 130	16
Обзорные статьи	11	9
Экспериментальные исследования	30	1

Итого для детального анализа было отобрано 26 источников. На основе этих источников были выделены методы обнаружения разладки, встречающиеся в сетевых задачах или рассматриваемые как релевантные для анализа временных рядов сетевых метрик.

1.2. Выводы из обзора

Систематический обзор показал, какие методы обнаружения разладки применяются к сетевым данным и какие из них целесообразно использовать как baseline для дальнейшего сравнения.

По результатам обзора были выделены основные методы обнаружения разладки, встречающиеся в сетевых задачах: Shewhart Control Chart, CUSUM, BOCPD, SDAR, ChangeFinder и SSA. Большинство найденных применений используют классические методы для одной сетевой метрики и опираются на предположения о независимости наблюдений, нормальности данных или заранее заданном характере изменения. Подробные описания рассмотренных методов, включая основные формулы, параметры и оценки вычислительной сложности, приведены в приложении 5.3.

В сетевых задачах чаще всего встречается адаптация классического CPD-метода к одной выбранной метрике. Например, TCP-NeuRoc использует BOCPD для адаптации TCP congestion control [5], а RoME-QCD использует CUSUM-подход для мониторинга 5G-сети по односторон-

ней задержке [7]. При этом готового специализированного решения для online-анализа транспортных метрик QUIC в рассмотренных источниках выявлено не было.

Из представленных методов были отобраны методы, отвечающие техническому заданию и используемые далее в качестве baseline для экспериментального сравнения:

- Shewhart Control Chart;
- Page CUSUM;
- Crosier CUSUM;
- Variance CUSUM;
- BOCPD.

Методы SDAR, ChangeFinder и SSA были рассмотрены в обзоре, однако не были включены в основной набор baseline-алгоритмов. Итогом раздела является набор методов, которые далее используются при экспериментальном сравнении, а также вывод о необходимости специализированного подхода и программного стенда для работы с транспортными метриками QUIC.

Глава 2. Архитектура программного стенда

В рамках работы был разработан программный стенд для экспериментальной оценки онлайн-алгоритмов обнаружения разладки. Стенд обеспечивает подготовку размеченных временных рядов, запуск алгоритмов, сохранение результатов выполнения и передачу этих результатов для дальнейшего анализа.

Программный стенд разрабатывался на основе опыта и отдельных заготовок проекта PySATL-CPD. Со стороны Huawei было разрешено опубликовать реализацию стенда, поэтому от коллектива разработчиков PySATL-CPD возник дополнительный запрос: стенд должен быть не частным решением только для одного эксперимента, а обобщаемой архитектурой, пригодной для дальнейшей интеграции в проект. Существующие заготовки не покрывали эти требования, поэтому часть архитектуры была переработана, а отдельные компоненты были реализованы заново.

Реализация доступна в открытом репозитории³.

2.1. Требования к программному стенду

Архитектура программного стенда определялась задачей воспроизводимой оценки онлайн-алгоритмов обнаружения разладки на временных рядах сетевых метрик. Для этого к стенду были предъявлены следующие требования:

а) Единое представление размеченных временных рядов.

Стенд должен хранить значения временного ряда вместе с эталонной разметкой. Разметка должна включать информацию о сегментах, режимах и точках разладки.

б) Идентификация объектов данных.

Каждый объект размеченных данных должен иметь устойчивый идентификатор, чтобы результаты запусков и последующий анализ можно было однозначно связать с исходным рядом.

в) Работа с наборами данных.

Стенд должен поддерживать загрузку временных рядов из датасета, чтение связанной с ними разметки и дополнительной аннотации.

г) Подготовка данных к экспериментам.

Стенд должен позволять преобразовывать временные ряды в формы, необходимые для сценариев бенчмаркинга, в том числе выделять фрагменты ряда и группировать их по типам переходов.

д) Поддержка синтетических данных.

Стенд должен позволять получать синтетические временные ряды с заранее известной разметкой, чтобы использовать их при разработке, отладке и сравнении алгоритмов.

е) Единый интерфейс запуска алгоритмов.

Алгоритмы обнаружения разладки должны запускаться через общий интерфейс и не зависеть от конкретного формата хранения данных.

ж) Сохранение результатов выполнения.

³<https://github.com/PySATL/rework-pysatl-cpd/>. История коммитов в опубликованной версии приведена к виду, не раскрывающему приватные детали проекта.

Результат запуска алгоритма должен сохраняться в форме, пригодной для повторного анализа без повторного запуска алгоритма.

з) **Воспроизводимость экспериментов.**

Стенд должен сохранять информацию о том, какие данные, алгоритмы и параметры использовались при запуске.

и) **Кеширование и экспорт результатов.**

Стенд должен поддерживать повторное использование ранее посчитанных запусков и сохранять итоговые результаты в формате, пригодном для последующей обработки.

2.2. Общая архитектура

Для выполнения перечисленных требований программный стенд был разделён на несколько слоёв:

- **data** — слой представления, загрузки и преобразования размеченных временных рядов;
- **core** — слой описания и запуска алгоритмов обнаружения разладки;
- **analysis** — слой анализа результатов запусков;
- **benchmarking** — слой организации экспериментальных запусков, кеширования и экспорта результатов.

Общая схема взаимодействия слоёв приведена на рисунке 3. Слой **data** предоставляет данные в едином внутреннем представлении. Слой **core** запускает алгоритм или детектор на подготовленных данных и сохраняет результат выполнения в трассу **DetectionTrace**. Слой **analysis** использует сохранённые трассы и эталонную разметку для анализа результатов. Слой **benchmarking** связывает остальные слои в экспериментальный сценарий: выбирает данные, запускает алгоритмы, обращается к кешу и сохраняет итоговые таблицы.

Далее кратко описаны слои стенда и их роль в выполнении требований.

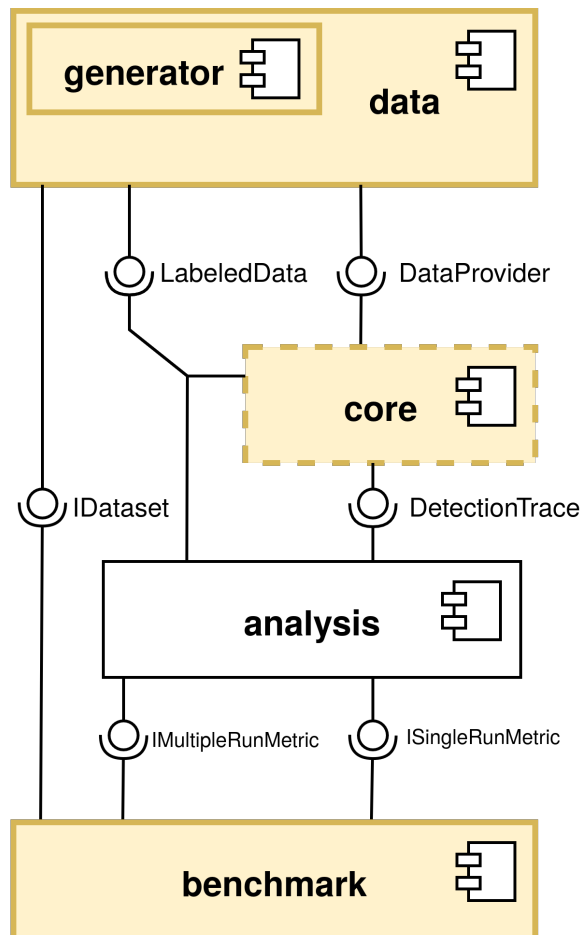


Рис. 3: Общая архитектура программного стенда

2.3. Архитектура слоя data

Слой **data** задаёт внутреннее представление данных, используемое остальными частями стенда. Базовый доступ к наблюдениям задаётся через абстракцию **DataProvider**. Размеченный временной ряд представлен сущностью **LabeledData**, которая объединяет значения наблюдений и эталонную разметку. Разметка сегментов вынесена в отдельную сущность **SegmentsLabeling**.

Для работы с наборами временных рядов используется абстракция **IDataset**. Она позволяет сценариям бенчмаркинга работать с датасетом как с набором размеченных рядов, не обращаясь напрямую к файловой структуре. Загрузка данных из файлов и их приведение к внутреннему представлению выполняются на уровне слоя **data**.

Внутри слоя также предусмотрены операции преобразования рядов. Они используются для подготовки данных к экспериментам: например,

для выделения фрагментов ряда и группировки фрагментов по типам переходов. Благодаря этому логика подготовки данных не смешивается с логикой запуска алгоритмов.

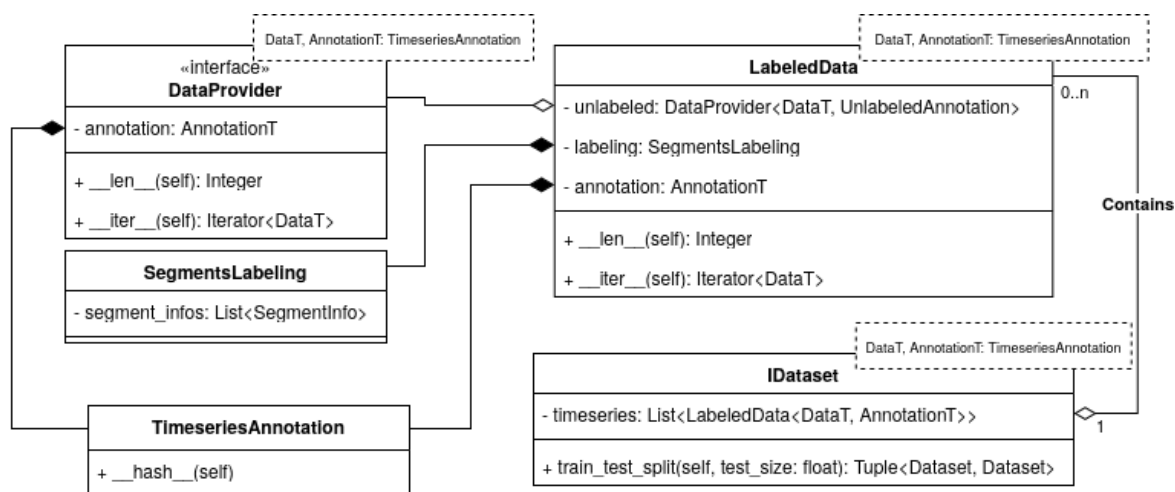


Рис. 4: Архитектура слоя data

В PySATL-CPD существовали только начальные заготовки функциональности для работы с данными. Они не поддерживали полноценную сущность идентифицируемых размеченных данных, не содержали необходимой функциональности для нарезки рядов, чтения рядов и обобщаемой работы с датасетами. Поэтому слой data был спроектирован и реализован автором настоящей работы с нуля.

2.4. Архитектура генератора синтетических данных

Генератор синтетических данных предназначен для получения временных рядов сетевых метрик с заранее известной эталонной разметкой. Для его реализации была разработана архитектура на основе дискретно-событийного моделирования. В такой архитектуре состояние моделируемой сети изменяется не непрерывно, а в моменты наступления событий.

В основной архитектуре стенда генератор рассматривается как источник размеченных данных для слоя data. Он позволяет получать ряды с заданными сценариями разладки, использовать их для отладки алгоритмов и формировать экспериментальные наборы данных. Подробная архитектура генератора, включая диаграмму классов и диаграмму

активностей моделирования, вынесена в приложение 5.3.

2.5. Архитектура слоя `core`

Слой `core` задаёт общий интерфейс запуска алгоритмов обнаружения разладки. В исходной версии PySATL-CPD существовала начальная архитектура этого слоя, однако она не соответствовала требованиям программного стенда. В рамках настоящей работы была предложена новая архитектура слоя `core`; её реализация выполнялась совместно с другими участниками проекта.

В новой архитектуре разделены алгоритм и детектор. `OnlineAlgorithm` отвечает за последовательную обработку наблюдений и вычисление статистики. `Detector` определяет, как результат работы алгоритма используется для принятия решения о разладке. Таким образом, вычисление статистики и логика принятия решения представлены разными сущностями.

Исходная абстракция `Solver` была переосмыслена и переименована в `Detector`. Это название точнее отражает назначение сущности: она не просто запускает вычисление, а формирует решение о наличии разладки на основе результата алгоритма.

Для описания алгоритмов и детекторов используются отдельные `description`-сущности. Они фиксируют название, параметры и способ построения соответствующего объекта. Такие описания используются при запуске экспериментов, сохранении конфигурации и последующем сопоставлении результатов с использованными параметрами.

Результат запуска детектора сохраняется в трассу `DetectionTrace`. Эта сущность отделяет выполнение алгоритма от последующего анализа: слой `core` формирует трассу, а дальнейшая обработка результата выполняется за пределами этого слоя.

2.6. Архитектура слоя `analysis`

Слой `analysis` отвечает за вычисление метрик качества и последующий анализ результатов запусков. Он получает сохранённые трассы

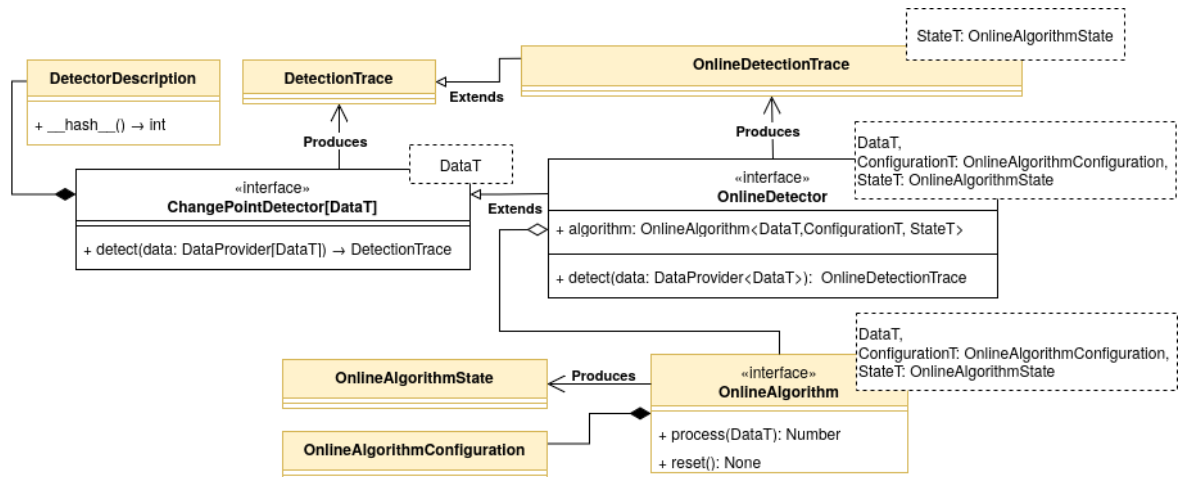


Рис. 5: Архитектура слоя core

DetectionTrace, сопоставляет их с эталонной разметкой и вычисляет показатели, используемые в экспериментальном сравнении: Precision, Recall, F_1 , PR-AUC, ADD и ARL. Кроме расчёта численных метрик, слой предоставляет инструменты для построения таблиц, графиков и визуализаций, необходимых для интерпретации результатов.

Слой analysis был разработан другим участником проекта. В настоящей работе он используется как компонент общей архитектуры стенда, но не относится к реализации, выполненной автором настоящей работы.

2.7. Архитектура слоя benchmarking

Слой benchmarking отвечает за организацию экспериментальных запусков. Он связывает слой данных, слой алгоритмов и слой анализа результатов. На этом уровне определяется, какие данные используются в эксперименте, какие детекторы запускаются и куда сохраняются результаты.

Основными сущностями слоя являются Benchmark, BenchmarkScenario, BenchmarkJob и BenchmarkRegistry. Benchmark управляет выполнением эксперимента. BenchmarkScenario описывает общий сценарий запуска. BenchmarkJob представляет отдельную задачу запуска. BenchmarkRegistry отвечает за хранение и повторное использование уже полученных результатов.

Слой **benchmarking** не реализует сами алгоритмы и не задаёт внутреннее представление временных рядов. Он использует слой **data** для получения подготовленных данных, слой **core** для запуска детекторов и слой **analysis** для последующей обработки результатов. Такое разделение позволяет изменять экспериментальные сценарии без изменения реализации алгоритмов и модели данных.

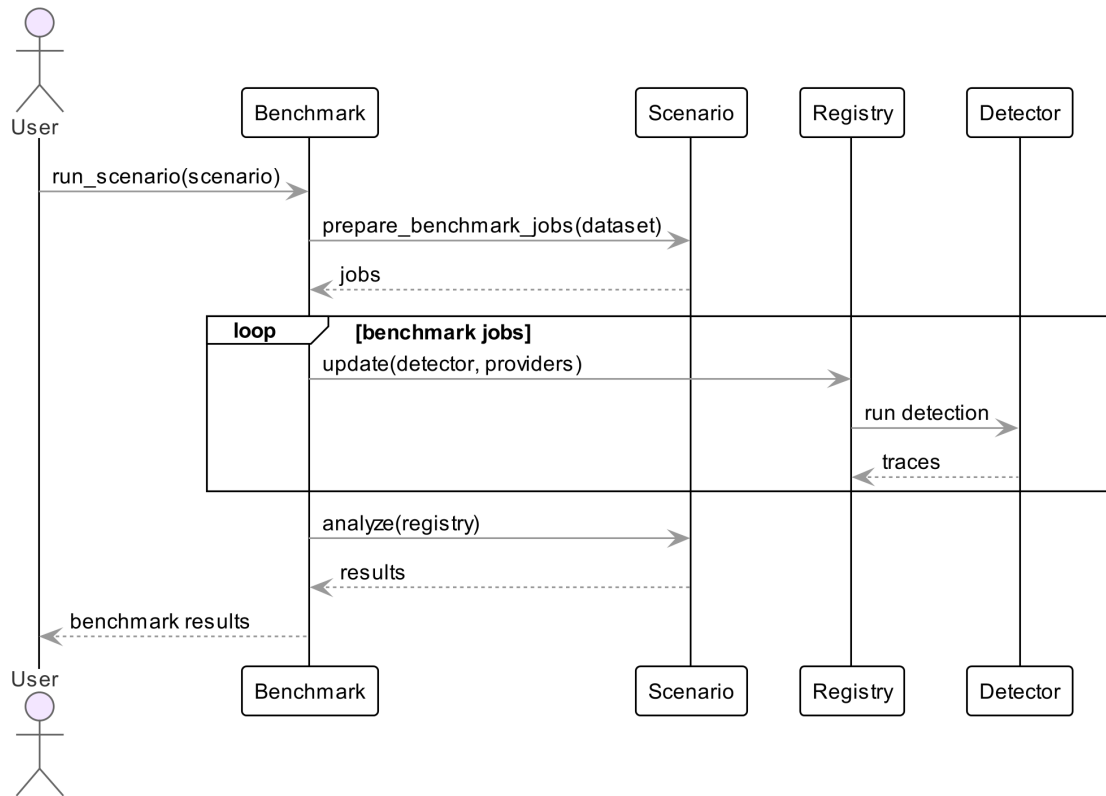


Рис. 6: Последовательность выполнения сценария бенчмаркинга

В PySATL-CPD существовала заготовка компонента бенчмаркинга, однако она не соответствовала требованиям настоящей работы и не содержала необходимого наполнения. Поэтому архитектура и реализация слоя **benchmarking** были выполнены автором настоящей работы с нуля.

Таким образом, программный стенд был построен как набор слоёв с разделённой ответственностью. В рамках настоящей работы автором были спроектированы и реализованы слой **data** и слой **benchmarking**; для слоя **core** автором была предложена новая архитектура, а реализация выполнялась совместно с другими участниками проекта. Генератор синтетических данных используется как компонент поддержки размеченных синтетических рядов. Слой **analysis** используется как часть

стенда, но был разработан не автором настоящей работы.

Глава 3. Особенности реализации программного стенда

В данном разделе описаны особенности реализации программного стенда, связанные с проведением экспериментов. Сначала приведён используемый технологический стек, затем рассмотрены два реализационных решения: поддержка воспроизводимости запусков и оптимизированные сценарии для онлайн-алгоритмов. Эти решения дополняют архитектуру, описанную в предыдущем разделе, и определяют способ хранения результатов, повторного использования вычислений и оценки алгоритмов при различных значениях порога.

3.1. Используемые технологии

Стенд реализован на языке Python, поскольку он используется в проекте PySATL-CPD и предоставляет развитую экосистему для анализа временных рядов и проведения вычислительных экспериментов. Для численных вычислений и работы с массивами применяются NumPy и SciPy, для табличной обработки данных — pandas, а для визуализации временных рядов, кривых качества и результатов экспериментов — Matplotlib и Seaborn. Такое разделение библиотек позволяет отделить реализацию алгоритмов и сценариев бенчмаркинга от последующей агрегации и визуального анализа результатов.

3.2. Воспроизводимость экспериментов

При сравнении алгоритмов обнаружения разладки необходимо сохранять не только итоговые значения метрик, но и условия, в которых эти значения были получены. Один и тот же алгоритм может запускаться на разных временных рядах, с разными параметрами алгоритма, с разными параметрами детектора и в разных экспериментальных сценариях. Поэтому результат запуска должен быть связан с полной конфигурацией эксперимента, а не только с названием алгоритма.

Для решения этой задачи в программном стенде используют

ся отдельные сущности для описания данных, алгоритмов, детекторов и запусков. Для временных рядов роль description-объекта выполняет `TimeseriesAnnotation`: эта сущность задаёт идентификатор и описание размеченных данных, по которым результат запуска связывается с исходным фрагментом ряда. Алгоритм описывается через `OnlineAlgorithmDescription`, где фиксируются публичное имя алгоритма и его конфигурация, а детектор — через `ChangePointDetectorDescription`, где фиксируются способ принятия решения о разладке и параметры этого решения.

Результат работы детектора сохраняется в объект `DetectionTrace`, который хранит трассу обработки временного ряда в формате, пригодном для последующего анализа. Поверх трассы строится `SingleRun`: он связывает сохранённый результат с размеченными данными, на которых был выполнен запуск. Для идентификации такого запуска используется хешируемый `SingleRunDescription`; если два запуска имеют одинаковое описание, то они соответствуют одной и той же комбинации данных, алгоритма, детектора и параметров сценария.

На уровне бенчмаркинга хранение результатов организовано через `BenchmarkRegistry`. Он работает как реестр соответствий между `SingleRunDescription` и `SingleRun`: перед выполнением нового запуска стенд проверяет, существует ли в реестре результат с тем же описанием, и либо переиспользует его, либо выполняет запуск и сохраняет новую трассу. Такой механизм используется и для локального кэширования, и для обмена результатами между участниками проекта, чтобы строить таблицы, графики и выполнять дополнительный анализ без повторного запуска алгоритмов.

3.3. Оптимизированные сценарии для онлайн алгоритмов

Отдельной особенностью реализации является поддержка оптимизированных сценариев для онлайн-алгоритмов. При экспериментальной оценке таких алгоритмов часто требуется исследовать качество не при одном фиксированном пороге, а как функцию от порога: например, сравнить число срабатываний, задержку обнаружения и производные

показатели качества при разных пороговых значениях.

Наивная реализация такого сценария состоит в том, чтобы для каждого значения порога заново запускать алгоритм на тех же данных. Для онлайн-алгоритмов это приводит к повторной последовательной обработке одного и того же временного ряда, хотя значения статистики алгоритма до применения порогового правила остаются неизменными.

Для устранения повторных запусков в стенде реализован `no-reset`-бенчмарк. В этом режиме `OnlineNoResetBenchmark` запускает онлайн-алгоритм через `NoResetOnlineDetector` без сброса состояния после потенциального срабатывания, а результат сохраняется в `NoResetDetectionTrace`. После этого сетка порогов задаётся отдельно через `ThresholdsRange`, а политики `NoResetPolicy` преобразуют сохранённую статистику в решения детектора для конкретных пороговых значений.

Такой сценарий используется при оценке онлайн-алгоритмов на бисегментах: алгоритм обрабатывает фрагмент один раз, после чего сохранённая статистика сопоставляется с эталонной точкой разладки при разных порогах. В результате `no-reset`-бенчмарк отделяет вычисление статистики онлайн-алгоритма, применение порогового правила и расчёт итоговых показателей качества, что уменьшает число повторных вычислений и упрощает последующий анализ результатов.

Глава 4. Прототип специализированного алгоритма обнаружения разладки

4.1. Общий подход

При разработке прототипа были приняты следующие предположения относительно природы наблюдаемого ряда.

- Между разладками поток транспортных метрик стационарен.
- При переходе из одного режима в другой структура зависимости наблюдаемого временного ряда может меняться, например может изменяться корреляция между соседними наблюдениями.
- Разладка в потоке транспортных метрик проявляется согласованно во всех метриках.

В данном разделе описывается прототип специализированного подхода CBCPD (Classifier Based Change Point Detection) для обнаружения разладки во временных рядах, состоящих из значений транспортных метрик сети. CBCPD рассматривается именно как подход: классификатор используется не как самостоятельное правило обнаружения разладки, а как способ построить отображение фрагментов ряда в пространство признаков режимов сети.

Основная идея подхода состоит в том, чтобы перейти от анализа отдельных значений отдельных метрик к анализу батчей, состоящих из последовательных наблюдений. Каждый батч рассматривается как элемент некоторого пространства $\mathcal{B}$, а изменение потока отслеживается через изменение средних значений функций, заданных на этом пространстве.

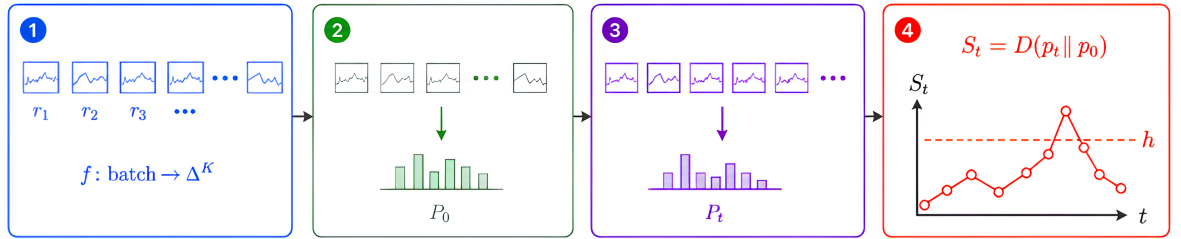


Рис. 7: Общая схема подхода CBCPD

Пусть входной поток наблюдений имеет вид

$$X = (\mathbf{x}_1, \mathbf{x}_2, \dots, \mathbf{x}_n, \dots), \quad \mathbf{x}_i \in \mathbb{R}^d, \quad (7)$$

где $\mathbf{x}_i$ — вектор сетевых метрик в момент времени i , а d — число используемых метрик. Из потока формируются батчи фиксированной длины b :

$$B_t = (\mathbf{x}_{t-b+1}, \mathbf{x}_{t-b+2}, \dots, \mathbf{x}_t). \quad (8)$$

Все такие батчи рассматриваются как элементы пространства батчей:

$$B_t \in \mathcal{B}. \quad (9)$$

На пространстве $\mathcal{B}$ задаётся набор функций

$$\varphi_1, \dots, \varphi_K : \mathcal{B} \rightarrow [0, 1], \quad (10)$$

образующий разбиение единицы:

$$\sum_{j=1}^K \varphi_j(B) = 1 \quad \text{для любого } B \in \mathcal{B}. \quad (11)$$

Тем самым каждому батчу сопоставляется вектор

$$\Phi(B) = (\varphi_1(B), \dots, \varphi_K(B)). \quad (12)$$

Подход не пытается напрямую определить, является ли отдельный батч моментом разладки. Вместо этого он сравнивает средний вектор $\Phi(B)$ на начальном участке потока со средним вектором, вычисленным по текущему скользящему окну. Если эти средние векторы существенно отличаются, это рассматривается как признак изменения статистического поведения потока.

Для применения подхода необходимо задать отображение

$$\varphi : \mathcal{B} \rightarrow [0, 1]^K, \quad \sum_{j=1}^K \varphi_j(B) = 1. \quad (13)$$

Это отображение сопоставляет каждому батчу вектор весов. Такие веса можно интерпретировать как разбиение единицы на пространстве батчей: каждая функция φ_j отвечает за одну компоненту описания, а сумма компонент для любого батча равна единице.

В общем случае разбиение может быть дискретным или мягким. При дискретном разбиении каждая функция φ_j принимает только значения 0 или 1:

$$\varphi_j(B) \in \{0, 1\}, \quad \sum_{j=1}^K \varphi_j(B) = 1. \quad (14)$$

Это означает, что каждый батч относится ровно к одной области пространства $\mathcal{B}$. Такой вариант можно получить, например, с помощью набора правил, кластеризации или классификатора, если использовать только итоговую предсказанную метку.

В прототипе используется мягкое разбиение. В этом случае батчу сопоставляется не один номер области, а вектор весов. Компоненты вектора показывают степень принадлежности батча к разным областям пространства. Такой вариант удобнее для обнаружения разладки, потому что сохраняет информацию о неуверенности модели и позволяет отслеживать постепенное изменение поведения потока. Конкретный способ построения мягкого разбиения относится к реализации прототипа и описан в следующем подразделе.

После построения отображения Φ алгоритм запускается на новом потоке наблюдений. Начальный участок потока используется как период подгонки. Его задача — получить базовый вектор средних значений функций φ_j .

Пусть за период подгонки было сформировано M полных батчей:

$$B_1, B_2, \dots, B_M. \quad (15)$$

Для каждого батча вычисляется вектор

$$q_i = \Phi(B_i), \quad i = 1, \dots, M. \quad (16)$$

Базовый вектор определяется как среднее значение этих векторов:

$$q_0 = \frac{1}{M} \sum_{i=1}^M q_i. \quad (17)$$

Покомпонентно:

$$q_0^{(j)} = \frac{1}{M} \sum_{i=1}^M \varphi_j(B_i), \quad j = 1, \dots, K. \quad (18)$$

Вектор q_0 описывает начальное поведение потока в координатах отображения Φ . При этом предполагается, что период подгонки со-

ответствует исходному состоянию потока. Если на этом участке уже присутствует разладка или переходный процесс, базовый вектор может быть искажён.

После завершения периода подгонки алгоритм переходит к режиму обнаружения. Для каждого нового батча B_t вычисляется

$$q_t = \Phi(B_t). \quad (19)$$

Для сглаживания отдельных значений используется скользящее окно размера $w = \text{window_size}$. Текущий средний вектор по окну задаётся как

$$\hat{q}_t = \frac{1}{w} \sum_{i=t-w+1}^t q_i. \quad (20)$$

Покомпонентно:

$$\hat{q}_t^{(j)} = \frac{1}{w} \sum_{i=t-w+1}^t \varphi_j(B_i). \quad (21)$$

Значение функции разладки определяется как расхождение между текущим средним вектором и базовым вектором:

$$s_t = D(\hat{q}_t, q_0), \quad (22)$$

где D — выбранная функция расхождения. В текущей версии прототипа используется евклидово расстояние между средними векторами вероятностей.

Если алгоритм должен самостоятельно возвращать момент обнаружения, то он может быть задан как

$$\tau = \inf \{t : s_t > h\}, \quad (23)$$

где h — порог обнаружения. Если же порог выбирается внешней процедурой экспериментальной оценки, алгоритм возвращает только последовательность значений s_t .

4.2. Реализация прототипа

Основными параметрами алгоритма являются:

- **batch_len** — длина батча b ;
- **длина периода подгонки** — число батчей M , используемых для построения базового вектора q_0 ;
- **window_size** — размер скользящего окна w ;
- **step_size** — шаг обновления функции разладки;
- **divergence** — функция расхождения D ;
- **отображение** Φ — способ построения разбиения единицы на пространстве батчей.

В реализации отображение Φ строится с помощью классификатора, обученного на исторических данных. Поэтому к параметрам реализации также относятся тип классификатора, параметры препроцессинга и способ формирования обучающей выборки. В рамках прототипа были реализованы два варианта подхода: CBCPD[LDA] с Linear Discriminant Analysis и CBCPD[RF] с Random Forest. Оба варианта используют общий конвейер формирования батчей, построения признаков, подгонки препроцессинга и вычисления функции разладки; различается только классификатор, задающий отображение Φ .

Для каждого батча в реализации выполняется конвейер обработки:

$$\text{batch} \rightarrow \text{features} \rightarrow \text{transform} \rightarrow \text{predict_proba}.$$

Сначала батч преобразуется в вектор признаков:

$$\mathbf{z} = \psi(B), \quad (24)$$

где ψ — процедура построения признакового описания батча. Затем применяется подогнанный препроцессинг:

$$\tilde{\mathbf{z}} = g(\mathbf{z}). \quad (25)$$

После этого обученный классификатор возвращает вероятности классов:

$$f(\tilde{\mathbf{z}}) = \mathbf{p} = \left(p^{(1)}, \dots, p^{(K)} \right), \quad p^{(j)} \geq 0, \quad \sum_{j=1}^K p^{(j)} = 1. \quad (26)$$

Этот вероятностный вектор используется как значение отображе-

ния φ :

$$\varphi(B) = f(g(\psi(B))). \quad (27)$$

Соответственно,

$$\varphi_j(B) = [f(g(\psi(B)))]_j. \quad (28)$$

Методика обучения классификатора устроена следующим образом. Сначала подготавливаются исторические временные ряды с метками режимов сети. В рассматриваемом сценарии такими режимами являются сбалансированный режим, недогруз и перегруз сети. Затем ряды разбиваются на сегменты и батчи фиксированной длины. Каждому батчу сопоставляется метка класса, соответствующая режиму сегмента, из которого был получен батч. После этого батчи преобразуются в признаковые векторы, и формируется обучающая выборка

$$\mathcal{D} = \{(\mathbf{z}_i, y_i)\}_{i=1}^N, \quad y_i \in \{1, \dots, K\}. \quad (29)$$

На этой выборке подгоняется препроцессинг g , после чего классификатор обучается на преобразованных векторах:

$$\tilde{\mathbf{z}}_i = g(\mathbf{z}_i). \quad (30)$$

В общем виде обучение можно записать как

$$\hat{f} = \arg \min_{f \in \mathcal{F}} \frac{1}{N} \sum_{i=1}^N L(f(\tilde{\mathbf{z}}_i), y_i), \quad (31)$$

где $\mathcal{F}$ — семейство моделей, а L — функция потерь. После обучения классификатор используется не для принятия решения о разладке напрямую. Его задача — построить отображение батчей в вероятностный симплекс; далее решение о разладке строится по функции s_t .

Ниже приведён псевдокод основной процедуры. Он отражает два этапа работы: период подгонки и последующее вычисление функции разладки.

Input: поток наблюдений X , длина батча b , длина периода подгонки M , размер окна w , отображение Φ , функция расхождения D

Output: последовательность значений функции разладки s_t

$Q_{\text{fit}} \leftarrow []$;

for $i \leftarrow 1$ **to** M **do**

$B_i \leftarrow$ следующий полный батч длины b из потока X ;

$q_i \leftarrow \Phi(B_i)$;

 добавить q_i в Q_{fit} ;

end

$q_0 \leftarrow \frac{1}{M} \sum_{q \in Q_{\text{fit}}} q$;

$W \leftarrow []$;

while *в потоке есть полный следующий батч* **do**

$B_t \leftarrow$ следующий полный батч длины b из потока X ;

$q_t \leftarrow \Phi(B_t)$;

 добавить q_t в W ;

if $|W| > w$ **then**

 удалить самый старый элемент из W ;

end

if $|W| = w$ **then**

$\hat{q}_t \leftarrow \frac{1}{w} \sum_{q \in W} q$;

$s_t \leftarrow D(\hat{q}_t, q_0)$;

 вернуть или сохранить s_t ;

end

end

Algorithm 1: Обнаружение разладки через разбиение единицы на пространстве батчей

В программном стенде оба варианта прототипа реализованы как online-алгоритмы, возвращающие последовательность значений функции разладки. Батчи формируются из последовательных наблюдений входного временного ряда с заданными параметрами `batch_len` и `step_size`. Для каждого батча вычисляется признаковое описание, применяется подогнанный препроцессинг, после чего LDA или Random Forest возвра-

щает вероятностный вектор $\Phi(B)$.

Период подгонки используется для вычисления базового вектора q_0 . Далее алгоритм поддерживает скользящее окно последних значений $\Phi(B)$, пересчитывает текущий средний вектор и сохраняет значение функции разладки. Порог обнаружения в экспериментальном сравнении выбирается внешней процедурой бенчмаркинга, поэтому реализация прототипа отделяет вычисление статистики от выбора конкретного порогового правила.

Вклад автора настоящей работы в этой части связан с формулировкой подхода CBSPD, интеграцией прототипа в программный стенд, запуском экспериментального сравнения и интерпретацией результатов. Подбор признаков и части параметров методов выполнялись совместно с участниками проекта и верифицировались заведующей кафедрой статистического моделирования Н. Э. Голяндиной.

Глава 5. Экспериментальное сравнение

В данном разделе проводится экспериментальное сравнение специализированного подхода к обнаружению разладки с baseline-методами, рассмотренными ранее. Подход далее обозначается как CBSPD (Classifier Based Change Point Detection). Цель сравнения состоит в том, чтобы проверить, показывает ли предложенный классификационный подход преимущество на модельных данных в контролируемой постановке.

Экспериментальное сравнение отвечает на следующие исследовательские вопросы:

- RQ1.** Может ли подход CBSPD превзойти baseline-методы по качеству обнаружения разладки?
- RQ2.** Можно ли на основе подхода CBSPD достигнуть меньшей задержки обнаружения при сопоставимом уровне ложных срабатываний?

5.1. Постановка эксперимента

Эксперимент проводился на модельных данных. Такой выбор связан с тем, что в рамках разработки прототипа сначала необходимо

проверить поведение алгоритма в контролируемых условиях: с точной эталонной разметкой, без дополнительных шумов, аномалий и эффектов, выходящих за рамки выбранной модели. Это позволяет получить более чистое сравнение алгоритмов и отдельно оценить, насколько специализированный подход использует информацию о рассматриваемом сценарии.

В качестве экспериментального сценария был выбран основной сценарий проекта: изменение отношения скорости отправки данных к доступной пропускной способности канала. Состояние *Normal* соответствует сбалансированному режиму, состояние *Overload* — ситуации, когда скорость отправки превышает доступную пропускную способность, а состояние *Underload* — ситуации недоиспользования доступной пропускной способности. Данный сценарий был согласован в рамках проекта с Huawei. Конкретные численные параметры сценария и параметры генерации не раскрываются в открытой части работы в силу ограничений проекта.

Для сравнения были сгенерированы 50 временных рядов стандартной длины 350 тыс. точек. Из них 10 рядов использовались для обучения, а 40 рядов — для тестирования. Такое соотношение было выбрано на основании опыта команды: при стандартной длине ряда порядка 350 тыс. точек значения PR-AUC стабилизируются при числе тестовых рядов порядка 30–40. Отдельное исследование чувствительности результата к выбору доли train/test не проводилось; это ограничение отдельно обсуждается в подразделе 5.3.

В сравнении участвовали baseline-методы, реализованные в программном стенде: Shewhart Control Chart, Page CUSUM, Variance CUSUM, многомерный Crosier CUSUM и BOCPD. Также в сравнение были включены два варианта CBCPD: вариант на основе Random Forest и вариант на основе LDA. RF и LDA были выбраны как относительно легковесные классификаторы; при этом Random Forest лучше подходит для учёта нелинейных зависимостей между сетевыми метриками. Поэтому именно вариант CBCPD на основе Random Forest рассматривается как основной classifier-вариант в итоговом сравнении.

Подбор признаков и параметров методов, за исключением значения `max_delay`, заданного техническим заданием, выполнялся другими участниками проекта и верифицировался заведующей кафедрой статистического моделирования Н. Э. Голяндиной. В рамках настоящей работы были выполнены запуск сравнительного эксперимента, агрегация результатов и их интерпретация.

Для оценки качества использовались метрики PR-AUC, F_1 , ADD (Average Detection Delay) и ARL. Их определения и методика вычисления приведены в обзорной главе настоящей работы. В данном разделе эти метрики используются для интерпретации результатов: PR-AUC характеризует качество обнаружения при изменении порога, F_1 отражает баланс точности и полноты при выбранном пороге, ADD показывает среднюю задержку обнаружения, а ARL характеризует среднее число наблюдений до первого ложного срабатывания.

5.2. Результаты эксперимента

Результаты сравнения приведены в таблице 2. В таблице сохранены технические имена конфигураций, использованные при запуске эксперимента. Для каждого метода указаны PR-AUC, значение F_1 и ADD.

Таблица 2: *Результаты экспериментального сравнения алгоритмов. Значение F_1 соответствует лучшему значению по сетке порогов, ADD вычисляется при том же пороге.*

Метод	PR-AUC	F_1	ADD
CBCPD[RF] [all]	1.000	1.000	32.200
Page CUSUM [rtt]	0.909	0.889	330.600
Shewhart Control Chart [rtt]	0.908	0.889	325.000
Variance CUSUM [rtt]	0.905	0.833	149.200
Crosier CUSUM [rtt, bw, rowd]	0.879	0.800	357.000
CBCPD[LDA] [all]	0.815	0.833	213.000
Shewhart Control Chart [bw]	0.711	0.667	329.000
BOCPD [rowd]	0.650	0.615	261.200

По результатам эксперимента вариант CBCPD на основе Random Forest показал наилучшие значения всех трёх рассматриваемых метрик:

PR-AUC равен 1.000, F_1 равен 1.000, а ADD составила 32.200 шага. Среди baseline-методов наибольшие значения PR-AUC показали Page CUSUM и Shewhart Control Chart по метрике RTT, однако их задержка оказалась существенно выше: 330.600 и 325.000 шага соответственно.

Отдельно можно отметить Variance CUSUM: этот метод имеет меньшую задержку, чем Page CUSUM и Shewhart Control Chart, но уступает им по F_1 и PR-AUC. Это показывает, что уменьшение задержки само по себе не является достаточным критерием качества: алгоритм должен одновременно обеспечивать приемлемое качество обнаружения и устойчивость к ложным срабатываниям. Вариант CBCPD на основе LDA уступил варианту на основе Random Forest, что согласуется с ожиданием о важности нелинейных зависимостей между сетевыми метриками.

Дополнительная качественная проверка проводилась по зависимости ADD от ARL. Эта зависимость позволяет сравнивать методы при разных уровнях допустимых ложных срабатываний. На рисунке 8 видно, что для переходов Normal $\rightarrow$ Underload и Normal $\rightarrow$ Overload вариант CBCPD на основе Random Forest сохраняет меньшую задержку обнаружения по сравнению с рассмотренными baseline-методами на большей части рассматриваемого диапазона ARL.

5.3. Обсуждение результатов

Таким образом, в рамках проведённого эксперимента на модельных данных специализированный подход CBCPD на основе Random Forest превзошёл рассмотренные baseline-методы как по качеству обнаружения разладки, так и по скорости обнаружения. Положительный результат состоит в том, что подход, использующий информацию о совместном поведении нескольких транспортных метрик, оказался эффективнее классических одномерных методов в выбранном сценарии. Это подтверждает исходную мотивацию работы: для транспортных метрик QUIC полезно использовать специализированное представление фрагментов ряда, а не только анализ отдельной метрики.

Ответ на RQ1 положительный: в выбранной экспериментальной постановке CBCPD[RF] превосходит рассмотренные baseline-методы по

PR-AUC и F_1 . Ответ на RQ2 также положительный: CBSPD[RF] обеспечивает меньшую ADD при сопоставимом анализе ложных срабатываний по ARL.

Основное ограничение эксперимента состоит в том, что сравнение проводилось на модельных, а не на реальных данных. Это соответствует цели этапа разработки прототипа, поскольку позволяет проверить алгоритм в чистой постановке с известной эталонной разметкой. Однако полученные результаты не следует автоматически переносить на реальный сетевой трафик: на практике могут возникать шумы, аномалии, неучтённые режимы работы сети и отличия распределений от модельных предположений.

Второе ограничение связано с выбором сценария и разбиения данных. Рассматривался основной сценарий проекта, согласованный с Huawei, но он не покрывает все возможные типы изменений в сетевых метриках. Кроме того, разбиение 10 рядов для обучения и 40 рядов для тестирования было выбрано на основании опыта команды, а не в результате отдельного исследования чувствительности. Это снижает полноту экспериментальной проверки, хотя и позволяет получить достаточно большой тестовый набор для оценки PR-AUC.

Отдельно отметим, что прямое переобучение на тестовые ряды в данной постановке не является основной угрозой: обучение и тестирование выполнялись на разных временных рядах. Более существенным ограничением является отсутствие проверки на реальном трафике и отсутствие более крупного эксперимента с несколькими независимыми сценариями генерации. В дальнейшем корректность выводов можно усилить за счёт расширения набора сценариев, проведения проверки на реальных данных и отдельного анализа чувствительности к параметрам алгоритмов.

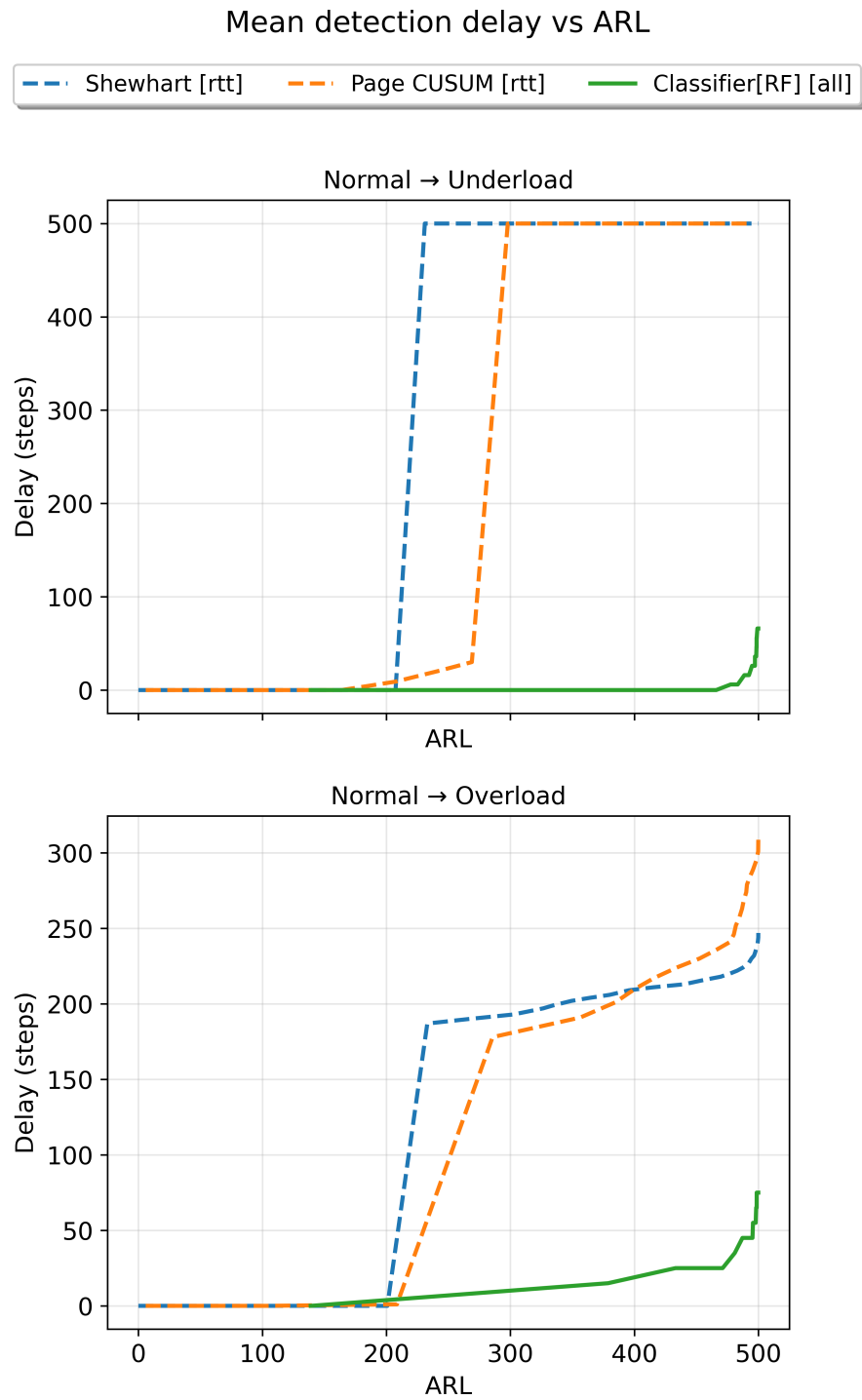


Рис. 8: Зависимость ADD от ARL для переходов Normal → Underload и Normal → Overload

Заключение

В ходе выполнения работы достигнуты следующие результаты, соотносимые с поставленными задачами:

а) Проведён систематический обзор методов обнаружения разладки в сетевых данных. По результатам обзора были выделены основные группы применяемых методов и выбран набор baseline-алгоритмов для дальнейшего сравнения: Shewhart Control Chart, Page CUSUM, Crosier CUSUM, Variance CUSUM и BOCPD.

б) Спроектирован и реализован программный стенд для подготовки данных, запуска онлайн-алгоритмов обнаружения разладки, сохранения трасс выполнения и воспроизводимого бенчмаркинга. В рамках стенда предусмотрена поддержка синтетических данных; генератор используется как один из источников размеченных временных рядов.

в) Разработан прототип специализированного подхода CBSPD для обнаружения разладки в транспортных метриках QUIC. Подход использует классификатор для построения отображения батчей наблюдений в вероятностный симплекс, после чего разладка определяется по изменению средних значений этого отображения.

г) Проведено экспериментальное сравнение CBSPD с baseline-методами на синтетических данных. В выбранной контролируемой постановке вариант CBSPD на основе Random Forest показал лучшие значения PR-AUC, F_1 и ADD среди рассмотренных методов.

Таким образом, поставленные задачи выполнены: в работе получены программные инструменты для воспроизводимого сравнения CPD-алгоритмов и прототип специализированного подхода, показывающий преимущество на модельных данных. Основными направлениями дальнейшей работы являются проверка подхода на реальном трафике, расширение набора сценариев генерации и дополнительное исследование чувствительности результатов к параметрам алгоритмов.

Приложение А. Подробные сведения о рассмотренных CPD-алгоритмах

В данном приложении приведены подробные описания методов обнаружения разладки, использованных при формировании набора baseline-алгоритмов. В основной части работы оставлены только выводы, необходимые для выбора методов сравнения.

Далее используются следующие обозначения: x_n — наблюдение в момент времени n , n — номер текущего наблюдения, w — размер скользящего окна, h — порог принятия решения.

Shewhart Control Chart

Shewhart Control Chart относится к классическим статистическим методам контроля изменения свойств процесса и часто рассматривается как простой baseline для задач обнаружения разладки [3, 8]. Метод отслеживает отклонение скользящего среднего последних w наблюдений от целевого значения μ_0 :

$$\bar{x}_n = \frac{1}{w} \sum_{i=n-w+1}^n x_i. \quad (32)$$

Разладка регистрируется, если отклонение превышает контрольные пределы, определяемые стандартным отклонением процесса σ и коэффициентом L :

$$|\bar{x}_n - \mu_0| > \frac{L\sigma}{\sqrt{w}}. \quad (33)$$

Основные параметры метода: целевое значение μ_0 , стандартное отклонение σ , коэффициент контрольной границы L и размер окна w . Вычислительная сложность составляет $O(1)$ по времени на одно наблюдение и $O(w)$ по памяти.

CUSUM

CUSUM-методы основаны на последовательном накоплении статистики, отражающей свидетельства в пользу изменения свойств наблю-

даемого процесса [10, 8]. В параметрической постановке используются распределения наблюдений до и после разладки. Пусть f_0 и f_1 обозначают плотности распределений до и после разладки соответственно. Тогда статистика может обновляться рекуррентно:

$$S_n = \max \left(0, S_{n-1} + \log \frac{f_1(x_n)}{f_0(x_n)} \right), \quad S_0 = 0. \quad (34)$$

Разладка фиксируется при превышении порога:

$$S_n > h. \quad (35)$$

Для вариантов, ориентированных на обнаружение сдвига среднего, статистика может строиться через накопление отклонений от целевого значения μ_0 :

$$S_n^+ = \max \left(0, S_{n-1}^+ + x_n - \mu_0 - \frac{\delta}{2} \right), \quad (36)$$

$$S_n^- = \max \left(0, S_{n-1}^- + \mu_0 - x_n - \frac{\delta}{2} \right), \quad (37)$$

где δ задаёт ожидаемый размер сдвига. Конкретные варианты CUSUM отличаются способом задания статистики: через отношение правдоподобий для распределений до и после разладки либо через накопление отклонений от целевого значения. В сетевых задачах CUSUM-подходы используются, например, для мониторинга 5G-сетей по односторонней задержке [7]. В дальнейшем сравнении рассматриваются отдельные представители этого семейства, включая Page CUSUM, Crosier CUSUM и Variance CUSUM. Для базовых рекуррентных вариантов сложность составляет $O(1)$ по времени и памяти.

BOCPD

Bayesian Online Changepoint Detection (BOCPD) вычисляет апостериорное распределение длины серии [1]:

$$P(r_n \mid x_{1:n}), \quad (38)$$

где r_n обозначает число наблюдений с момента последней разладки. Обновление распределения выполняется при поступлении каждого нового наблюдения x_n с учётом априорного распределения момента разладки и выбранной модели данных. В общем виде шаг обновления можно записать как

$$P(r_n \mid x_{1:n}) \propto \sum_{r_{n-1}} P(r_n \mid r_{n-1}) P(x_n \mid r_{n-1}, x_{1:n-1}) P(r_{n-1} \mid x_{1:n-1}). \quad (39)$$

Решение о наличии разладки принимается на основе этого распределения. Например, можно использовать критерий

$$1 - P(r_n = n - 1 \mid x_{1:n}) > h. \quad (40)$$

ВОСПД применялся и в сетевом контексте: в TCP-NeuRos он используется совместно с обучением с подкреплением для адаптации контроля перегрузки [5]. Основные параметры метода: априорное распределение разладки, модель данных и порог принятия решения h . Базовая вычислительная сложность составляет $O(n^2)$ по времени и памяти; при усечении истории она может быть снижена до $O(w^2)$.

SDAR и ChangeFinder

Метод SDAR адаптивно обучает авторегрессионную модель $AR(p)$ с дисконтирующим фактором r , за счёт которого более старые наблюдения постепенно теряют влияние на текущую модель [11]. Авторегрессионная модель порядка p имеет вид

$$x_n = \sum_{i=1}^p a_i x_{n-i} + \varepsilon_n, \quad (41)$$

где a_i — параметры модели, а ε_n — ошибка модели. Разладка обнаруживается как аномалия в скользящем среднем от логарифмической правдоподобности новых наблюдений. Соответствующую статистику

можно записать как

$$A_n = \frac{1}{w} \sum_{i=n-w+1}^n (-\log p_{i-1}(x_i)), \quad (42)$$

где $p_{i-1}(x_i)$ — вероятность наблюдения x_i относительно модели, построенной по предыдущим наблюдениям. Основные параметры метода: порядок авторегрессии p , фактор дисконтирования r и размер окна усреднения w . В наивной реализации вычислительная сложность составляет $O(p^3)$ по времени и $O(p^2)$ по памяти. Аппаратная реализация ChangeFinder на сетевой карте FPGA рассмотрена в работе [4].

SSA

Методы на основе сингулярного спектрального анализа используются для построения сигнального подпространства по скользящему окну наблюдений и рассматриваются в обзорах CPD-методов как один из вариантов анализа временных рядов [6, 3]. Статистика разладки вычисляется как сумма квадратов расстояний векторов тестовой выборки до построенного подпространства:

$$D_{n,l,p,q} = \sum_{j=p+1}^q \left(\|X_j^{(n)}\|^2 - \|U^T X_j^{(n)}\|^2 \right), \quad (43)$$

где U — матрица, столбцы которой задают выбранные собственные векторы сигнального подпространства, а $X_j^{(n)}$ — векторы тестовой выборки. Затем на эту статистику может накладываться CUSUM-статистика. Основные параметры метода: размер окна w , размер лага k , число собственных векторов l , параметры тестовой выборки (p, q) и порог h . Вычислительная сложность составляет $O(wk \log k)$ по времени на наблюдение и $O(w^2)$ по памяти.

Приложение В. Архитектура генератора синтетических данных

Генератор синтетических данных предназначен для получения временных рядов сетевых метрик с заранее известной эталонной разметкой. Для его реализации была разработана архитектура на основе дискретно-событийного моделирования. В такой архитектуре состояние моделируемой сети изменяется не непрерывно, а в моменты наступления событий.

Генератор состоит из трёх основных типов сущностей:

- **Generator** — координирует выполнение моделирования, управляет активной конфигурацией сети и переключает конфигурации в моменты разладки;
- **Actor** — базовая сущность для активных компонентов модели сети;
- **Event** — сообщение о событии, которое передаётся между акторами и изменяет состояние моделирования.

В качестве акторов используются компоненты, моделирующие элементы сетевого взаимодействия: отправитель, буфер и маршрутизатор. Например, **Sender** отвечает за генерацию пакетов, **OWDBuffer** моделирует одностороннюю задержку и буферизацию, а **Router** моделирует обработку пакетов сетевым узлом с ограниченной пропускной способностью.

Для задания случайных времён обработки пакетов используется интерфейс **DistributionFactory**. Он позволяет менять статистические свойства моделируемых задержек через параметры распределений. Разладка в синтетических данных задаётся изменением конфигурации моделирования: в заданный момент времени изменяются параметры распределений или структурные параметры сети, например пропускная способность, размер буфера или задержка распространения.

Процедура моделирования представлена на диаграмме активностей на рисунке 10. Генератор обрабатывает события в хронологическом порядке: выбирает ближайшее событие, передаёт его соответствующему актору, обновляет состояние модели и добавляет в очередь новые собы-

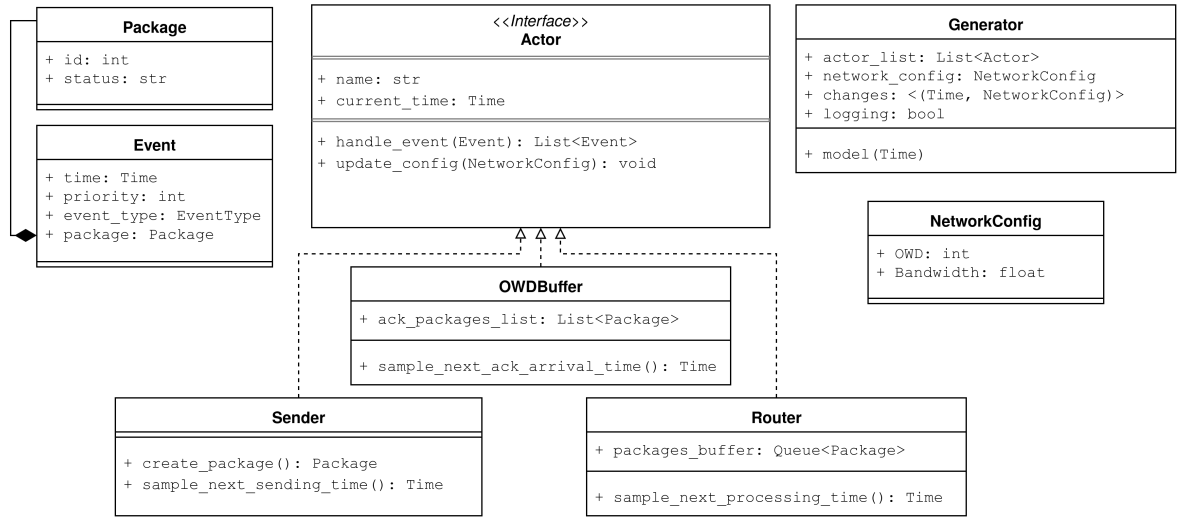


Рис. 9: Архитектура генератора синтетических данных

тия, возникшие в результате обработки. Если в процессе моделирования достигается момент разладки, генератор меняет активную конфигурацию сети, после чего дальнейшие события обрабатываются уже при новых параметрах моделирования.

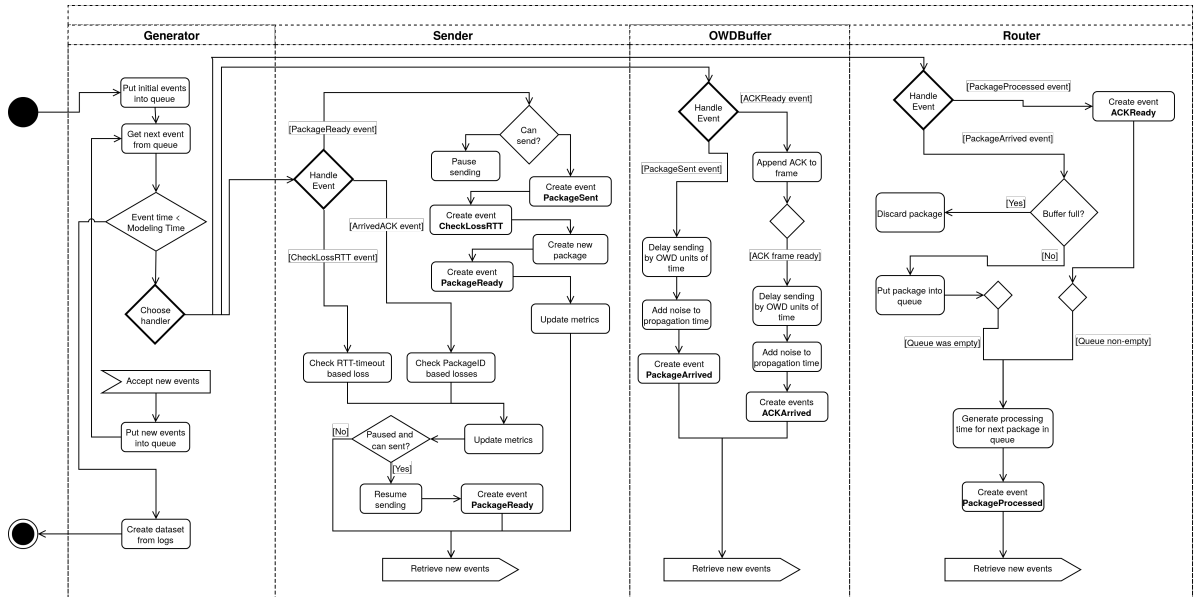


Рис. 10: Диаграмма активностей генератора синтетических данных

Список литературы

- [1] Ryan Prescott Adams и David J. C. MacKay. *Bayesian Online Changepoint Detection*. 2007. arXiv: 0710.3742 [stat.ML]. URL: <https://arxiv.org/abs/0710.3742>.

- [2] Gerrit J. J. van den Burg и Christopher K. I. Williams. *An Evaluation of Change Point Detection Algorithms*. arXiv.org. 13 марта 2020. URL: <https://arxiv.org/abs/2003.06222v3> (дата обр. 28.11.2025).
- [3] Muktesh Gupta, Rajesh Wadhvani и Akhtar Rasool. «Comprehensive analysis of change-point dynamics detection in time series data: A review». В: *Expert Systems with Applications* 248 (авг. 2024), с. 123342. ISSN: 09574174. DOI: 10.1016/j.eswa.2024.123342. URL: <https://linkinghub.elsevier.com/retrieve/pii/S0957417424002070> (дата обр. 14.08.2025).
- [4] Takuma Iwata и др. «Accelerating Online Change-Point Detection Algorithm Using 10 GbE FPGA NIC». В: *Euro-Par 2018: Parallel Processing Workshops*. Под ред. Gabriele Mencagli и др. Т. 11339. Series Title: Lecture Notes in Computer Science. Cham: Springer International Publishing, 2019, с. 506—517. ISBN: 978-3-030-10548-8. DOI: 10.1007/978-3-030-10549-5_40. URL: https://link.springer.com/10.1007/978-3-030-10549-5_40 (дата обр. 16.11.2025).
- [5] Wenzhong Li и др. «TCP-NeuRoc: Neural Adaptive TCP Congestion Control With Online Changepoint Detection». В: *IEEE Journal on Selected Areas in Communications* 39.8 (авг. 2021), с. 2461—2475. ISSN: 0733-8716, 1558-0008. DOI: 10.1109/JSAC.2021.3087247. URL: <https://ieeexplore.ieee.org/document/9448107/> (дата обр. 16.11.2025).
- [6] Yixiao Li и др. *A Review of Changepoint Detection Models*. 20 авг. 2019. DOI: 10.48550/arXiv.1908.07136. arXiv: 1908.07136[stat]. URL: <http://arxiv.org/abs/1908.07136> (дата обр. 28.11.2025).
- [7] Simon Lindståhl, Alexandre Proutiere и Andreas Johnsson. «RoME-QCD: Robust and Measurement Efficient Quickest Change Detection in 5G Networks». В: 2024 8th Network Traffic Measurement and Analysis Conference (TMA). IEEE, 2024, с. 1—11. ISBN: 3-903176-64-8.

- [8] Bernadin Namono и др. «Online change detection techniques in time series: An overview». В: *2019 IEEE International Conference on Prognostics and Health Management (ICPHM)*. 2019 IEEE International Conference on Prognostics and Health Management (ICPHM). San Francisco, CA, USA: IEEE, июнь 2019, с. 1—10. ISBN: 978-1-5386-8357-6. DOI: 10.1109/ICPHM.2019.8819394. URL: <https://ieeexplore.ieee.org/document/8819394/> (дата обр. 27.11.2025).
- [9] Wenqin Shao и др. «One-to-one matching of RTT and path changes». В: *2017 29th International Teletraffic Congress (ITC 29)*. Т. 1. IEEE, 2017, с. 196—204. ISBN: 0-9883045-3-8. DOI: 10.23919/ITC.2017.8064356.
- [10] Haoyun Wang и Yao Xie. «Sequential change-point detection: Computation versus statistical performance». В: *Wiley Interdisciplinary Reviews: Computational Statistics* 16.1 (1 янв. 2024). Publisher: John Wiley & Sons, Ltd, e1628. ISSN: 1939-0068. DOI: 10.1002/wics.1628. URL: <https://wires.onlinelibrary.wiley.com/doi/10.1002/wics.1628> (дата обр. 27.11.2025).
- [11] Kenji Yamanishi и Jun-ichi Takeuchi. «A unifying framework for detecting outliers and change points from non-stationary time series data». В: *Proceedings of the eighth ACM SIGKDD international conference on Knowledge discovery and data mining*. KDD02. ACM, июль 2002, с. 676—681. DOI: 10.1145/775047.775148. URL: <http://dx.doi.org/10.1145/775047.775148>.